Part of the AgentCon World Tour by the Global AI Community

AI Agents Developer Conference

agentcon.city/rome

Microsoft

global AI community

.NET CODE

KEY2 NEXT

BC Soft consulenza sistemi informativi

GitHub Agentic Workflows: Continuous AI per la manutenzione del software che non dorme mai

Giulio Sciarappa – Azure MVP Observability & Well-Architected Framework

agentcon.city/rome

Made Possible by

I will always choose a lazy person to do a difficult job because a lazy person will find an easy way to do it.

Bill Gates?

Frank Gilbreth Sr.

Clarence Bleicher

General Kurt von Hammerstein-Equord

Il problema: automazione che non scala

Issue triage manuale

Ogni issue richiede attenzione umana per label, priorità, assegnazione

CI failure investigation

Scavare nei log per capire perché la build sì è rotta, ogni volta

Docs sempre obsoleta

La documentazione invecchia più velocemente del codice

Refactoring accantonato

Il tech debt cresce perché nessuno ha tempo di affrontarlo

"Queste attività sono importanti, ripetitive, e richiedono comprensione del contesto… insomma, il candidato perfetto per gli agenti AI."

Da CI/CD a Continuous AI

CI/CD Tradizionale

- Deterministico
- If/then rigidi in YAML
- Un trigger → un'azione fissa
- Non comprende il contesto
- Manutenzione script costosa

Continuous AI

- Adattivo e contestuale
- Istruzioni in linguaggio naturale
- L'agente capisce e decide
- Si adatta a scenari diversi
- Automazione definita in Markdown

Augmenta la CI/CD deterministica senza sostituirla

GitHub Agentic Workflows in 60 secondi

Markdown-first

Scrivi automazione in .md invece che in YAML complesso

Security-first

Read-only di default, safe outputs, sandboxing, firewall

Multi-engine

Copilot CLI, Claude Code, OpenAI Codex — stesso formato

Open source

MIT license, sviluppato da GitHub Next + Microsoft Research

GitHub-native

Compilato in Actions workflow, triggers, log, secrets — tutto nativo

Event & schedule

Issue, PR, cron, dispatch, slash commands nei commenti

Anatomia di un Agentic Workflow

.github/workflows/daily-report.md

```
---
on:
  schedule: daily
permissions:
  contents: read
  issues: read
safe-outputs:
  create-issue:
    title-prefix: "[status]"
    labels: [report]
---

## Daily Issues Report

Create an upbeat daily status
report for the team.
```

FRONTMATTER

Trigger, permessi,
safe-outputs
tutto dichiarativo in YAML

ISTRUZIONI

Linguaggio naturale
che l'agente AI
esegue come task

Il ciclo: Write → Compile → Run

1

WRITE

Crea un file .md in
.github/workflows/
con frontmatter + istruzioni

2

COMPILE

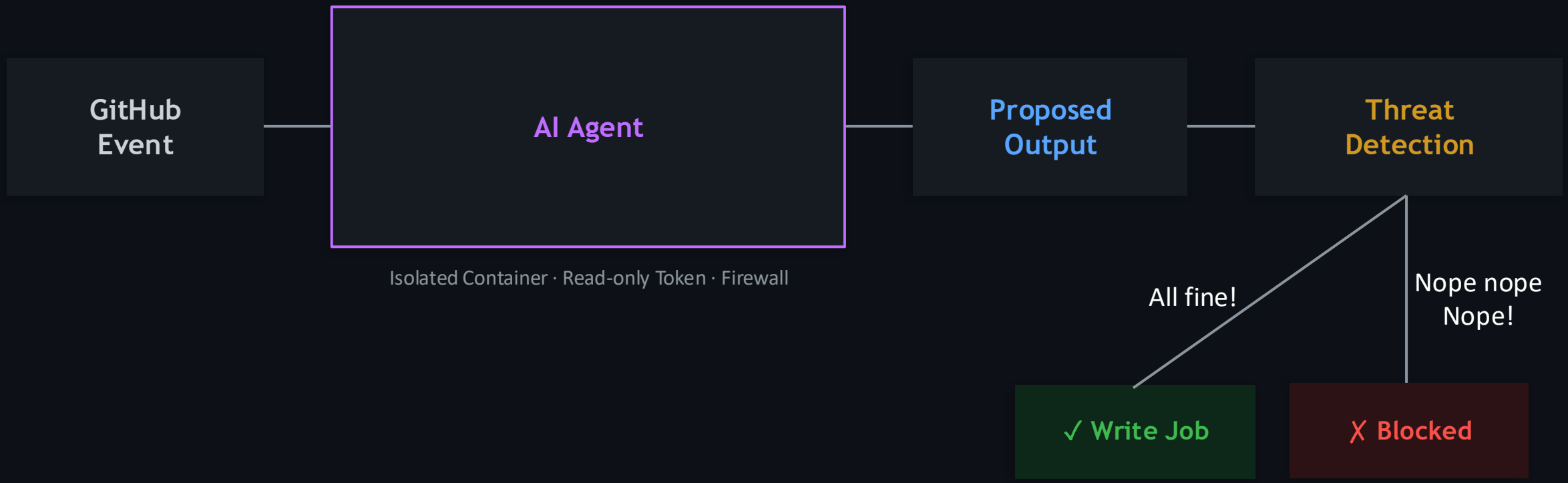
gh aw compile genera
un .lock.yml hardened
con guardrails integrati

3

RUN

GitHub Actions esegue
il workflow: l'agente AI
opera nel sandbox

Security Architecture: Defense-in-Depth



I 5 livelli di sicurezza

- Read-only tokens
- Credential isolation
- Network firewall
- Tool allowlist
- Output sanitization

I 5 livelli nel dettaglio

1 

Read-only tokens

L'agente riceve un token scoped read-only. Non può pushare, creare PR o cancellare file.

2 

Credential isolation

I secrets e i write token esistono solo in job separati, dopo la review. L'agente non ha nulla da rubare.

3 

Network firewall (AWF)

Tutto il traffico passa per un proxy Squid con domain allowlist esplicita. Nessun egress non autorizzato.

4 

Tool allowlisting

Solo gli strumenti dichiarati nel frontmatter sono disponibili. MCP Gateway centralizzato per il routing.

5 

Output sanitization

Ogni output passa per threat detection AI-powered. Se qualcosa è sospetto, il workflow fallisce.

Safe Outputs: write senza write token

L'agente AI non ha mai accesso diretto in scrittura.
I safe outputs sono azioni pre-approvate, dichiarate nel frontmatter, validate ed eseguite in un job separato con token scoped.

`create-issue`	Crea issue con prefix obbligatorio e label predefiniti
`create-pr`	Crea PR con branch naming e auto-close constraints
`add-comment`	Aggiunge commenti a issue/PR con sanitizzazione
`add-label`	Applica label da un set pre-approvato
`close-issue`	Chiude issue solo se matcha criteri dichiarati

Il pattern: l'agente propone → il sistema valida → un job separato esegue

Engine supportati

Stesso formato workflow, engine intercambiabili

GitHub Copilot CLI

Default

Integrazione nativa, accesso al contesto GitHub,
modello ottimizzato per codice

Claude by Anthropic

Sperimentale

Claude Code come coding agent,
forte su ragionamento e analisi complessa

OpenAI Codex

Sperimentale

Codex CLI agent,
focus su generazione e trasformazione codice

+ MCP Tools: GitHub MCP Server, browser automation, web search, custom MCP servers

Gallery: workflow pronti all'uso

Repo Assist

Triage, bug fix, commenti e assistant multiuso

Issue Triage

Label, priorità e assegnazione automatica

CI Doctor

Monitora CI e investiga fallimenti automaticamente

Metrics & Analytics

Report giornalieri, trend e salute dei workflow

Continuous Refactoring

Semplificazione e miglioramento del codice quotidiano

Security Scanning

Alert triage, compliance e monitoraggio sicurezza

Continuous Docs

Manutenzione documentazione e coerenza continua

Workflow Optimizer (Q)

Analizza e ottimizza i tuoi agentic workflow

Il .lock.yml: cosa genera il compiler

20 righe di Markdown → workflow Actions production-grade con sicurezza integrata

daily-report.md

```
---
on:
  schedule: daily
permissions:
  contents: read
safe-outputs:
  create-issue:
    title-prefix: "[status]"
---

## Daily Issues Report
Create an upbeat daily
status report.
```

gh aw compile

daily-report.lock.yml

```
jobs:
  agent:
    runs-on: ubuntu-latest
    container: ghcr.io/.../sandbox
    permissions:
      contents: read  # read-only!
    env:
      AWF_PROXY: squid-firewall:3128
      AWF_ALLOWED_DOMAINS: ...

  threat-detect:
    needs: agent
    # AI-powered output validation

  write:
    needs: threat-detect
    permissions:
      issues: write  # scoped token
```

agent — Read-only, sandboxed, firewalled

threat-detect — Valida output prima del write

write — Scoped token, solo safe-outputs

MCP Tools & Gateway

Gli strumenti che l'agente può usare, governati da un gateway centralizzato

Solo i tool dichiarati nel frontmatter sono accessibili

Il Gateway fa routing e logging centralizzato di ogni tool call

Tool call sospette vengono bloccate prima dell'esecuzione

Quando NON usare Agentic Workflows

Non tutto va delegato a un agente. Sapere dove NON usarli è importante quanto sapere dove usarli.

Task deterministici puri

Linting, formatting, build, test — GitHub Actions classico è più veloce, economico e prevedibile

Dati sensibili senza audit

Operazioni su PII, credenziali, database di produzione — serve controllo umano e compliance trail

Decisioni irreversibili

Deploy in produzione, merge su main, cancellazioni — l'agente propone, l'umano decide

Output che richiede accuratezza 100%

Documentazione legale, report finanziari, comunicazioni ufficiali . Ricorda, l'AI può sbagliare

La regola d'oro: se puoi scriverlo in un if/then deterministico, non ti serve un agente.

DEMO

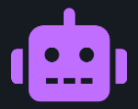
Dal Markdown alla Pull Request automatica

1. gh aw init → scaffolding del workflow .md
2. gh aw compile → generazione del .lock.yml
3. Push + trigger → l'agente lavora nel sandbox
4. Review della PR generata automaticamente

Key Takeaways

Markdown-first: automazione descritta in linguaggio naturale,
compilata in GitHub Actions workflow hardened

Security by design: 5 livelli di protezione, read-only di default,
safe outputs, sandboxing e network firewall



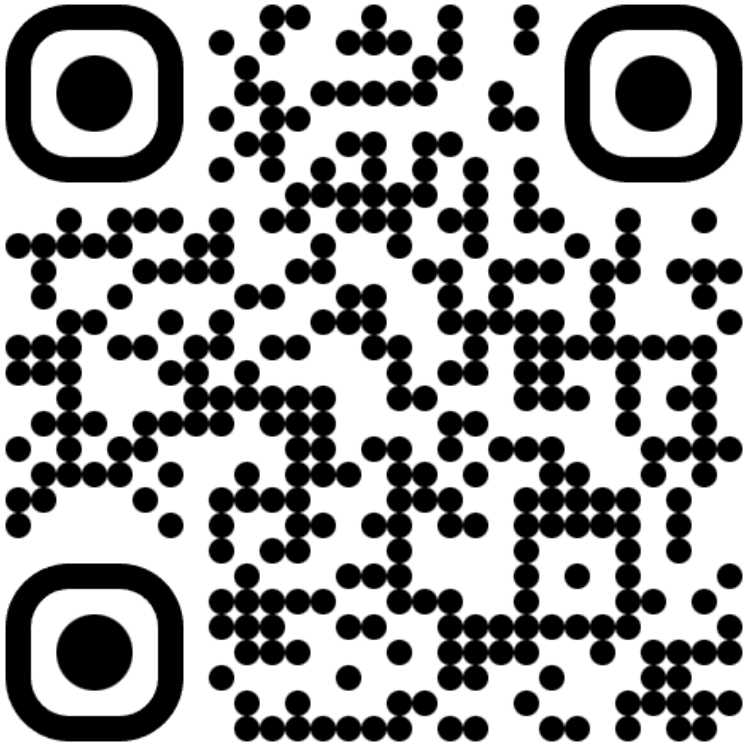
Engine-agnostic: Copilot, Claude, Codex, stesso formato,
stesse guardrails, engine intercambiabile

Continuous AI: non va a sostituire la CI/CD deterministica,
la augmenta con intelligenza adattiva e contestuale

Grazie!

Domande?

Join the AI Conversation on Discord



gaic.io/discord